[1]:
```
import numpy as np
from scipy import linalg
from scipy.sparse.linalg import eigs
from scipy.sparse.linalg import cg
from scipy import signal
```

0.0.1 Defining some arrays for different numpy operations

[2]:
```
a = np.arange(1., 37.).reshape(6, 6)

A = np.array([[4., 1., 1.],
              [1., 5., 1.],
              [1., 1., 6.]])

B = np.array([[2., 1., 0.],
              [1., 3., 1.],
              [0., 1., 2.]])

v = np.array([1., 2., 3.])
```

[3]:
```
a.ndim
```

[3]: 2

[4]:
```
np.size(a)
```

[4]: 36

[5]:
```
np.shape(a)
```

[5]: (6, 6)

[6]:
```
a.shape[1]
```

[6]: 6

[7]:
```
new_arr = np.array([[1., 2., 3.], [4., 5., 6.]])
```

[8]:
```
ax = np.array([[1, 2],
               [3, 4]])

bx = np.array([[5],
               [6]])

cx = np.array([[7, 8]])

dx = np.array([[9]])
```

[9]:
```
ax[-1]
```

[9]:
```
array([3, 4])
```

[10]:
```
A[1, 2]
```

[10]:
```
np.float64(1.0)
```

[11]:
```
ax[1, :]
```

[11]:
```
array([3, 4])
```

[12]:
```
a[0:5]
```

[12]:
```
array([[ 1.,  2.,  3.,  4.,  5.,  6.],
       [ 7.,  8.,  9., 10., 11., 12.],
       [13., 14., 15., 16., 17., 18.],
       [19., 20., 21., 22., 23., 24.],
       [25., 26., 27., 28., 29., 30.]])
```

[13]:
```
a[-5:]
```

[13]:
```
array([[ 7.,  8.,  9., 10., 11., 12.],
       [13., 14., 15., 16., 17., 18.],
       [19., 20., 21., 22., 23., 24.],
       [25., 26., 27., 28., 29., 30.],
       [31., 32., 33., 34., 35., 36.]])
```

[14]:
```
a[0:3, 4:9]
```

[14]:
```
array([[ 5.,  6.],
       [11., 12.],
       [17., 18.]])
```

[15]:
```
a[np.ix_([1, 3, 4], [0, 2])]
```

[15]:
```
array([[ 7.,  9.],
       [19., 21.],
```

```
       [25., 27.]])
```

[16]:
```
a[2:21:2,:]
```

[16]:
```
array([[13., 14., 15., 16., 17., 18.],
       [25., 26., 27., 28., 29., 30.]])
```

[17]:
```
a[::2, :]
```

[17]:
```
array([[ 1.,  2.,  3.,  4.,  5.,  6.],
       [13., 14., 15., 16., 17., 18.],
       [25., 26., 27., 28., 29., 30.]])
```

[18]:
```
a[::-1,:]
```

[18]:
```
array([[31., 32., 33., 34., 35., 36.],
       [25., 26., 27., 28., 29., 30.],
       [19., 20., 21., 22., 23., 24.],
       [13., 14., 15., 16., 17., 18.],
       [ 7.,  8.,  9., 10., 11., 12.],
       [ 1.,  2.,  3.,  4.,  5.,  6.]])
```

[19]:
```
a[np.r_[:len(a),0]]
```

[19]:
```
array([[ 1.,  2.,  3.,  4.,  5.,  6.],
       [ 7.,  8.,  9., 10., 11., 12.],
       [13., 14., 15., 16., 17., 18.],
       [19., 20., 21., 22., 23., 24.],
       [25., 26., 27., 28., 29., 30.],
       [31., 32., 33., 34., 35., 36.],
       [ 1.,  2.,  3.,  4.,  5.,  6.]])
```

[20]:
```
a.T
```

[20]:
```
array([[ 1.,  7., 13., 19., 25., 31.],
       [ 2.,  8., 14., 20., 26., 32.],
       [ 3.,  9., 15., 21., 27., 33.],
       [ 4., 10., 16., 22., 28., 34.],
       [ 5., 11., 17., 23., 29., 35.],
       [ 6., 12., 18., 24., 30., 36.]])
```

[21]:
```
a.conj().transpose()
```

[21]:
```
array([[ 1.,  7., 13., 19., 25., 31.],
       [ 2.,  8., 14., 20., 26., 32.],
       [ 3.,  9., 15., 21., 27., 33.],
       [ 4., 10., 16., 22., 28., 34.],
       [ 5., 11., 17., 23., 29., 35.],
```

```
       [ 6., 12., 18., 24., 30., 36.]])
```

[22]:
```
A @ B
```

[22]:
```
array([[ 9.,  8.,  3.],
       [ 7., 17.,  7.],
       [ 3., 10., 13.]])
```

[23]:
```
A * B
```

[23]:
```
array([[ 8.,  1.,  0.],
       [ 1., 15.,  1.],
       [ 0.,  1., 12.]])
```

[24]:
```
A/B
```

```
C:\Users\ghimi\AppData\Local\Temp\ipykernel_15820\713994841.py:1:
RuntimeWarning: divide by zero encountered in divide
  A/B
```

[24]:
```
array([[2.        , 1.        ,        inf],
       [1.        , 1.66666667, 1.        ],
       [       inf, 1.        , 3.        ]])
```

[25]:
```
a**3
```

[25]:
```
array([[1.0000e+00, 8.0000e+00, 2.7000e+01, 6.4000e+01, 1.2500e+02,
        2.1600e+02],
       [3.4300e+02, 5.1200e+02, 7.2900e+02, 1.0000e+03, 1.3310e+03,
        1.7280e+03],
       [2.1970e+03, 2.7440e+03, 3.3750e+03, 4.0960e+03, 4.9130e+03,
        5.8320e+03],
       [6.8590e+03, 8.0000e+03, 9.2610e+03, 1.0648e+04, 1.2167e+04,
        1.3824e+04],
       [1.5625e+04, 1.7576e+04, 1.9683e+04, 2.1952e+04, 2.4389e+04,
        2.7000e+04],
       [2.9791e+04, 3.2768e+04, 3.5937e+04, 3.9304e+04, 4.2875e+04,
        4.6656e+04]])
```

[26]:
```
(a > 0.5)
```

[26]:
```
array([[ True,  True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True,  True]])
```

[27]:
```
np.nonzero(a > 0.5)
```

[27]:
```
(array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3,
        3, 3, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5]),
 array([0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5, 0, 1, 2, 3,
        4, 5, 0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5]))
```

[28]:
```
a[:,np.nonzero(v > 0.5)[0]]
```

[28]:
```
array([[ 1.,  2.,  3.],
       [ 7.,  8.,  9.],
       [13., 14., 15.],
       [19., 20., 21.],
       [25., 26., 27.],
       [31., 32., 33.]])
```

[29]:
```
A[:, v.T > 0.5]
```

[29]:
```
array([[4., 1., 1.],
       [1., 5., 1.],
       [1., 1., 6.]])
```

[30]:
```
a[a < 0.5]=0
a
```

[30]:
```
array([[ 1.,  2.,  3.,  4.,  5.,  6.],
       [ 7.,  8.,  9., 10., 11., 12.],
       [13., 14., 15., 16., 17., 18.],
       [19., 20., 21., 22., 23., 24.],
       [25., 26., 27., 28., 29., 30.],
       [31., 32., 33., 34., 35., 36.]])
```

[31]:
```
a * (a > 0.5)
```

[31]:
```
array([[ 1.,  2.,  3.,  4.,  5.,  6.],
       [ 7.,  8.,  9., 10., 11., 12.],
       [13., 14., 15., 16., 17., 18.],
       [19., 20., 21., 22., 23., 24.],
       [25., 26., 27., 28., 29., 30.],
       [31., 32., 33., 34., 35., 36.]])
```

[32]:
```
a[:] = 3
a
```

[32]:
```
array([[3., 3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3., 3.],
```

```
       [3., 3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3., 3.]])
```

[33]:
```
y = a.copy()
y
```

[33]:
```
array([[3., 3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3., 3.],
       [3., 3., 3., 3., 3., 3.]])
```

[34]:
```
y = a[1, :].copy()
y
```

[34]:
```
array([3., 3., 3., 3., 3., 3.])
```

[35]:
```
y = a.flatten()
y
```

[35]:
```
array([3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
       3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
       3., 3.])
```

[36]:
```
np.arange(1., 11.)
```

[36]:
```
array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
```

[37]:
```
np.arange(10.)
```

[37]:
```
array([0., 1., 2., 3., 4., 5., 6., 7., 8., 9.])
```

[38]:
```
np.arange(1.,11.)[:, np.newaxis]
```

[38]:
```
array([[ 1.],
       [ 2.],
       [ 3.],
       [ 4.],
       [ 5.],
       [ 6.],
       [ 7.],
       [ 8.],
       [ 9.],
       [10.]])
```

[39]:
```
np.zeros((3, 4))
```

[39]:
```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

[40]:
```
np.zeros((3, 4, 5))
```

[40]:
```
array([[[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]],

       [[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]],

       [[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]]])
```

[41]:
```
np.ones((3, 4))
```

[41]:
```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

[42]:
```
np.eye(3)
```

[42]:
```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

[43]:
```
np.diag(a)
```

[43]:
```
array([3., 3., 3., 3., 3., 3.])
```

[44]:
```
np.diag(v, 0)
```

[44]:
```
array([[1., 0., 0.],
       [0., 2., 0.],
       [0., 0., 3.]])
```

[45]:
```
from numpy.random import default_rng
rng = default_rng(42)
rng.random((3, 4))
```

[45]:
```
array([[0.77395605, 0.43887844, 0.85859792, 0.69736803],
       [0.09417735, 0.97562235, 0.7611397 , 0.78606431],
       [0.12811363, 0.45038594, 0.37079802, 0.92676499]])
```

[46]:
```
np.linspace(1,3,4)
```

[46]:
```
array([1.        , 1.66666667, 2.33333333, 3.        ])
```

[47]:
```
np.mgrid[0:9.,0:6.]
```

[47]:
```
array([[[0., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 1.],
        [2., 2., 2., 2., 2., 2.],
        [3., 3., 3., 3., 3., 3.],
        [4., 4., 4., 4., 4., 4.],
        [5., 5., 5., 5., 5., 5.],
        [6., 6., 6., 6., 6., 6.],
        [7., 7., 7., 7., 7., 7.],
        [8., 8., 8., 8., 8., 8.]],

       [[0., 1., 2., 3., 4., 5.],
        [0., 1., 2., 3., 4., 5.],
        [0., 1., 2., 3., 4., 5.],
        [0., 1., 2., 3., 4., 5.],
        [0., 1., 2., 3., 4., 5.],
        [0., 1., 2., 3., 4., 5.],
        [0., 1., 2., 3., 4., 5.],
        [0., 1., 2., 3., 4., 5.],
        [0., 1., 2., 3., 4., 5.]]])
```

[48]:
```
np.ix_(np.r_[0:9.],np.r_[0:6.])
```

[48]:
```
(array([[0.],
        [1.],
        [2.],
        [3.],
        [4.],
        [5.],
        [6.],
        [7.],
        [8.]]),
 array([[0., 1., 2., 3., 4., 5.]]))
```

[49]:
```
np.meshgrid([1,2,4],[2,4,5])
```

[49]:
```
(array([[1, 2, 4],
        [1, 2, 4],
        [1, 2, 4]]),
```

```
array([[2, 2, 2],
       [4, 4, 4],
       [5, 5, 5]]))
```

[50]:
```
np.ix_([1,2,4],[2,4,5])
```

[50]:
```
(array([[1],
        [2],
        [4]]),
 array([[2, 4, 5]]))
```

[51]:
```
np.tile(a, (2, 3))
```

[51]:
```
array([[3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.],
       [3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3., 3.,
        3., 3.]])
```

[52]:
```
np.concatenate((A,B),1)
```

[52]:
```
array([[4., 1., 1., 2., 1., 0.],
       [1., 5., 1., 1., 3., 1.],
       [1., 1., 6., 0., 1., 2.]])
```

[53]:
```
a.max()
```

[53]:
```
np.float64(3.0)
```

[54]:
```
a.max(0)
```

[54]:
```
array([3., 3., 3., 3., 3., 3.])
```

[55]:
```
a.max(1)
```

[55]:
```
array([3., 3., 3., 3., 3., 3.])
```

[56]:
```
np.maximum(A, B)
```

[56]:
```
array([[4., 1., 1.],
       [1., 5., 1.],
       [1., 1., 6.]])
```

[57]:
```
np.sqrt(v @ v)
```

[57]:
```
np.float64(3.7416573867739413)
```

[58]:
```
np.logical_and(A,B)
```

[58]:
```
array([[ True,  True, False],
       [ True,  True,  True],
       [False,  True,  True]])
```

[59]:
```
np.logical_or(A,B)
```

[59]:
```
array([[ True,  True,  True],
       [ True,  True,  True],
       [ True,  True,  True]])
```

[60]:
```
A = np.array([1.2, 0.0, 3.4])
B = np.array([2.1, 1.0, 0.0])

(A > 0.5) & (B > 0.5)
```

[60]:
```
array([ True, False, False])
```

[61]:
```
A_int = A.astype(int)
B_int = B.astype(int)

A_int | B_int
```

[61]:
```
array([3, 1, 3])
```

[62]:
```
A = np.array([
    [1, 2],
    [3, 4]
])
```

```
B = np.array([
    [6, 6],
    [6, 6]
])

linalg.inv(A)
```

[62]:
```
array([[-2. ,  1. ],
       [ 1.5, -0.5]])
```

[63]:
```
linalg.pinv(a)
```

[63]:
```
array([[0.00925926, 0.00925926, 0.00925926, 0.00925926, 0.00925926,
        0.00925926],
       [0.00925926, 0.00925926, 0.00925926, 0.00925926, 0.00925926,
        0.00925926],
       [0.00925926, 0.00925926, 0.00925926, 0.00925926, 0.00925926,
        0.00925926],
       [0.00925926, 0.00925926, 0.00925926, 0.00925926, 0.00925926,
        0.00925926],
       [0.00925926, 0.00925926, 0.00925926, 0.00925926, 0.00925926,
        0.00925926],
       [0.00925926, 0.00925926, 0.00925926, 0.00925926, 0.00925926,
        0.00925926]])
```

[64]:
```
np.linalg.matrix_rank(a)
```

[64]:
```
np.int64(1)
```

[65]:
```
linalg.lstsq(A, B)
```

[65]:
```
(array([[-6., -6.],
        [ 6.,  6.]]),
 array([], dtype=float64),
 np.int64(2),
 array([5.4649857 , 0.36596619]))
```

[66]:
```
U, S, Vh = linalg.svd(a); V = Vh.T

print(U, S, Vh)
```

```
[[-4.08248290e-01  9.12870929e-01  2.73607193e-16  1.46575282e-17
   3.90867419e-17  8.79451693e-17]
 [-4.08248290e-01 -1.82574186e-01 -2.23606798e-01 -6.33830541e-01
  -4.17288177e-01 -4.17288177e-01]
 [-4.08248290e-01 -1.82574186e-01 -2.23606798e-01  7.67661083e-01
  -2.83457635e-01 -2.83457635e-01]
```

```
 [-4.08248290e-01 -1.82574186e-01 -2.23606798e-01 -6.69152707e-02
  -1.49627094e-01  8.50372906e-01]
 [-4.08248290e-01 -1.82574186e-01 -2.23606798e-01 -6.69152707e-02
   8.50372906e-01 -1.49627094e-01]
 [-4.08248290e-01 -1.82574186e-01  8.94427191e-01 -1.90039842e-16
  -2.66224590e-16 -2.79816208e-16]] [1.80000000e+01 9.06493304e-16
1.76394661e-31 0.00000000e+00
 0.00000000e+00 0.00000000e+00] [[-4.08248290e-01 -4.08248290e-01
-4.08248290e-01 -4.08248290e-01
  -4.08248290e-01 -4.08248290e-01]
 [-9.12870929e-01  1.82574186e-01  1.82574186e-01  1.82574186e-01
   1.82574186e-01  1.82574186e-01]
 [-0.00000000e+00 -8.94427191e-01  2.23606798e-01  2.23606798e-01
   2.23606798e-01  2.23606798e-01]
 [ 0.00000000e+00  0.00000000e+00 -5.00000000e-01 -1.66666667e-01
  -1.66666667e-01  8.33333333e-01]
 [ 0.00000000e+00 -6.02523696e-17 -5.00000000e-01 -1.66666667e-01
   8.33333333e-01 -1.66666667e-01]
 [ 0.00000000e+00 -4.01682464e-17 -5.00000000e-01  8.33333333e-01
  -1.66666667e-01 -1.66666667e-01]]
```

[67]:
```
a = np.array([
    [4, 2],
    [2, 3]
])

b = np.array([
    [2, 0],
    [0, 1]
])

L = linalg.cholesky(a, lower=True)
L
```

[67]:
```
array([[2.        , 0.        ],
       [1.        , 1.41421356]])
```

[68]:
```
D,V = linalg.eig(a)
```

[69]:
```
D, V = linalg.eig(a, b)

print(D)   # generalized eigenvalues
print(V)   # eigenvectors
```

```
[1.+0.j 4.+0.j]
[[ 0.70710678  0.4472136 ]
 [-0.70710678  0.89442719]]
```

[70]:
```
D,V = eigs(a, k=3)

print(D, V)
```

```
[5.56155281+0.j 1.43844719+0.j] [[ 0.78820544 -0.61541221]
 [ 0.61541221  0.78820544]]
```

```
C:\Users\ghimi\AppData\Local\Temp\ipykernel_15820\1187404391.py:1:
RuntimeWarning: k >= N - 1 for N * N square matrix. Attempting to use
scipy.linalg.eig instead.
  D,V = eigs(a, k=3)
```

[71]:
```
Q,R = linalg.qr(a)

print(Q,R)
```

```
[[-0.89442719 -0.4472136 ]
 [-0.4472136   0.89442719]] [[-4.47213595 -3.13049517]
 [ 0.          1.78885438]]
```

[72]:
```
P,L,U = linalg.lu(a)

print(P, L, U)
```

```
[[1. 0.]
 [0. 1.]] [[1.  0. ]
 [0.5 1. ]] [[4. 2.]
 [0. 2.]]
```

[73]:
```
cg
```

[73]:
```
<function scipy.sparse.linalg._isolve.iterative.cg(A, b, x0=None, *, rtol=1e-05,
atol=0.0, maxiter=None, M=None, callback=None)>
```

[74]:
```
np.fft.fft(a)
```

[74]:
```
array([[ 6.+0.j,  2.+0.j],
       [ 5.+0.j, -1.+0.j]])
```

[75]:
```
np.fft.ifft(a)
```

[75]:
```
array([[ 3. +0.j,  1. +0.j],
       [ 2.5+0.j, -0.5+0.j]])
```

[76]:
```
np.sort(a)
```

[76]:
```
array([[2, 4],
       [2, 3]])
```

[77]:
```
np.sort(a, axis=1)
```

[77]:
```
array([[2, 4],
       [2, 3]])
```

[78]:
```
I = np.argsort(a[:, 0]); b = a[I,:]
I
```

[78]:
```
array([1, 0])
```

[79]:
```
Z = np.array([
    [1, 1],
    [1, 2],
    [1, 3],
    [1, 4]
])

y = np.array([2, 3, 5, 6])

x = linalg.lstsq(Z, y)

print(x)
```

```
(array([0.5, 1.4]), array(0.2), np.int64(2), array([5.77937881, 0.77380911]))
```

[80]:
```
x = np.array([1, 2, 3, 4, 5, 6])
q = 2

y = signal.resample(x, int(np.ceil(len(x) / q)))

print(y)
```

```
[2.5 2.5 5.5]
```

[81]:
```
np.unique(a)
```

[81]:
```
array([2, 3, 4])
```

[82]:
```
a.squeeze()
```

[82]:
```
array([[4, 2],
       [2, 3]])
```

[ ]: